

빅데이터 분석 정의서

2025. 12. 12

기능명	루틴 우선순위 예측 및 최적화 기능
1. 데이터 준비	
데이터 정의	우선순위 루틴 추천 모델 학습용 더미데이터
데이터 획득 방법	모델 학습에 필요한 데이터를 확보하기 위해 ThinQ API의 로그와 기기 로그 특성, 외부 API 데이터 형식을 반영해 난수를 설정하고 이를 기반 더미 데이터 5000건을 생성했다. 난수 설정 코드 <pre> # ----- # 1) 루틴 정보 (Routines 기반) # ----- df["ROUTINE_TYPE"] = np.random.choice([0, 1, 2, 3, 4], size=N, p=[0.25, 0.2, 0.2, 0.2, 0.15]) df["RUN_MINUTES"] = np.random.randint(5, 121, size=N) # 5~120분 df["SCHEDULE_TYPE"] = rng.choice([0, 1, 2, 3, 4], size=N, p=[0.15, 0.45, 0.20, 0.15, 0.05]) df["PREFERRED_TIME"] = rng.choice([6, 8, 10, 12, 15, 19, 21, 23], size=N) # 사용자가 좋아하는 시간 # ----- # 2) 현재 시각 컨텍스트 (Flask의 now() 역할) # ----- exec_times = pd.date_range("2025-03-01", periods=N, freq="H") df["EXEC_HOUR"] = rng.integers(6, 23, size=N) df["EXEC_DOW"] = rng.integers(0, 7, size=N) # ----- # 날씨 / 환경 df["TEMPERATURE"] = rng.normal(20, 7, size=N).clip(-5, 35) # 대충 -5~35도 df["HUMIDITY"] = rng.integers(20, 90, size=N) # 20~90% df["WEATHER"] = rng.integers(0, 4, size=N) # 0:맑음~3:눈 df["PM25"] = rng.integers(10, 120, size=N) df["PM10"] = rng.integers(20, 200, size=N) # 실행 예상 시간 (분) df["RUN_MINUTES"] = rng.integers(10, 61, size=N) # 실제 실행 시간 (예상치 근처로 노이즈) df["RUN_TIME"] = (df["RUN_MINUTES"] * rng.normal(1.0, 0.2, size=N)).clip(df["RUN_MINUTES"] * 0.5, df["RUN_MINUTES"] * 1.8) </pre>

지도 학습을 위한 라벨링 규칙

```
def compute_priority(row):
    s = 0

    # 1) 스케줄 가중치
    if row["SCHEDULE_TYPE"] == 1: # DAILY
        s += 2.75
    elif row["SCHEDULE_TYPE"] == 2: # WEEKLY
        s += 1.5
    elif row["SCHEDULE_TYPE"] == 3: # MONTHLY
        s += 1
    else: #CUSTOM 아무때나 하기만 하면될때
        s += 0.5

    # 2) 선호시간 vs 현재시간
    gap = abs(row["PREFERRED_TIME"] - row["EXEC_HOUR"])
    if gap <= 1:
        s += 3
    elif gap <= 3:
        s += 1
    else:
        s -= 1

    # 3) 추천 기반 실행 여부
    if row["RECOMMENDED_FLAG"] == 1:
        s += 1

    # 4) 미세먼지 영향을 강하게
    if (row["PM25"] > 80 or row["PM10"] > 150) and row["ROUTINE_TYPE"] == 0:
        s += 2

    # 5) (선택) 오래 안 한 루틴 우대
    if "DAYS_SINCE_LAST" in row:
        if row["DAYS_SINCE_LAST"] >= 5:
            s += 2
        elif row["DAYS_SINCE_LAST"] >= 2:
            s += 1

    return s

df["RULE_BASE_SCORE"] = df.apply(compute_priority, axis=1)

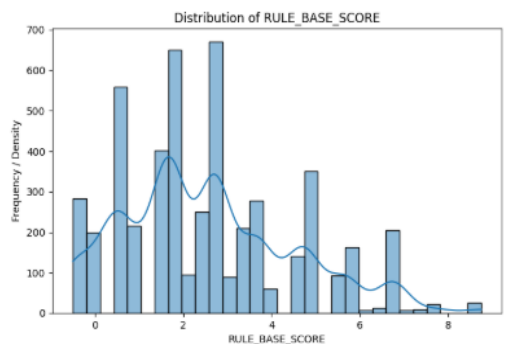
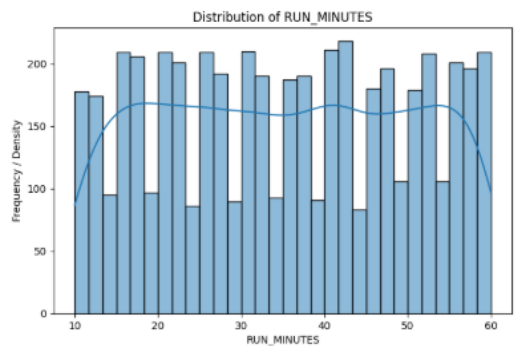
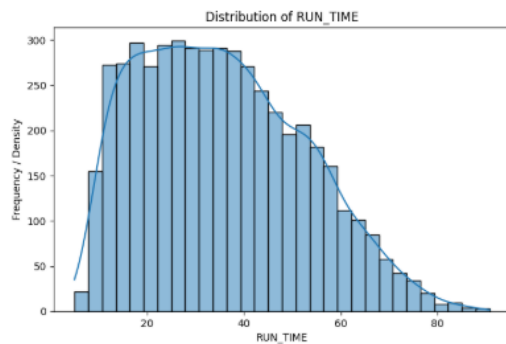
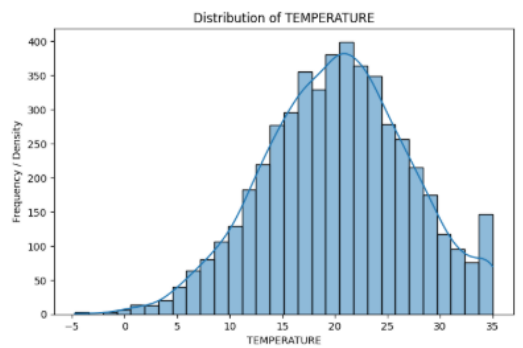
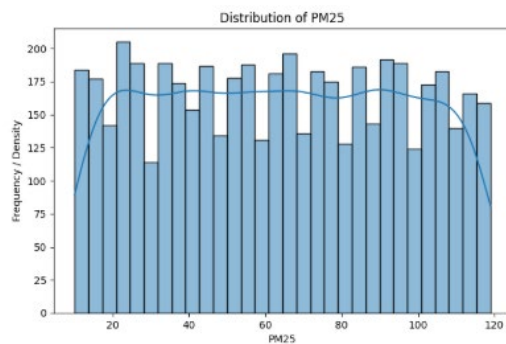
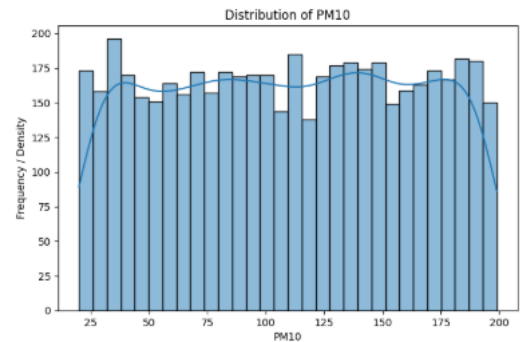
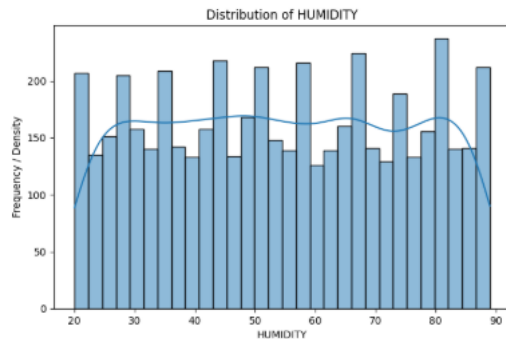
noise = rng.normal(0, 0.5, size=N)
df["PRIORITY_SCORE"] = df["RULE_BASE_SCORE"] + noise
```

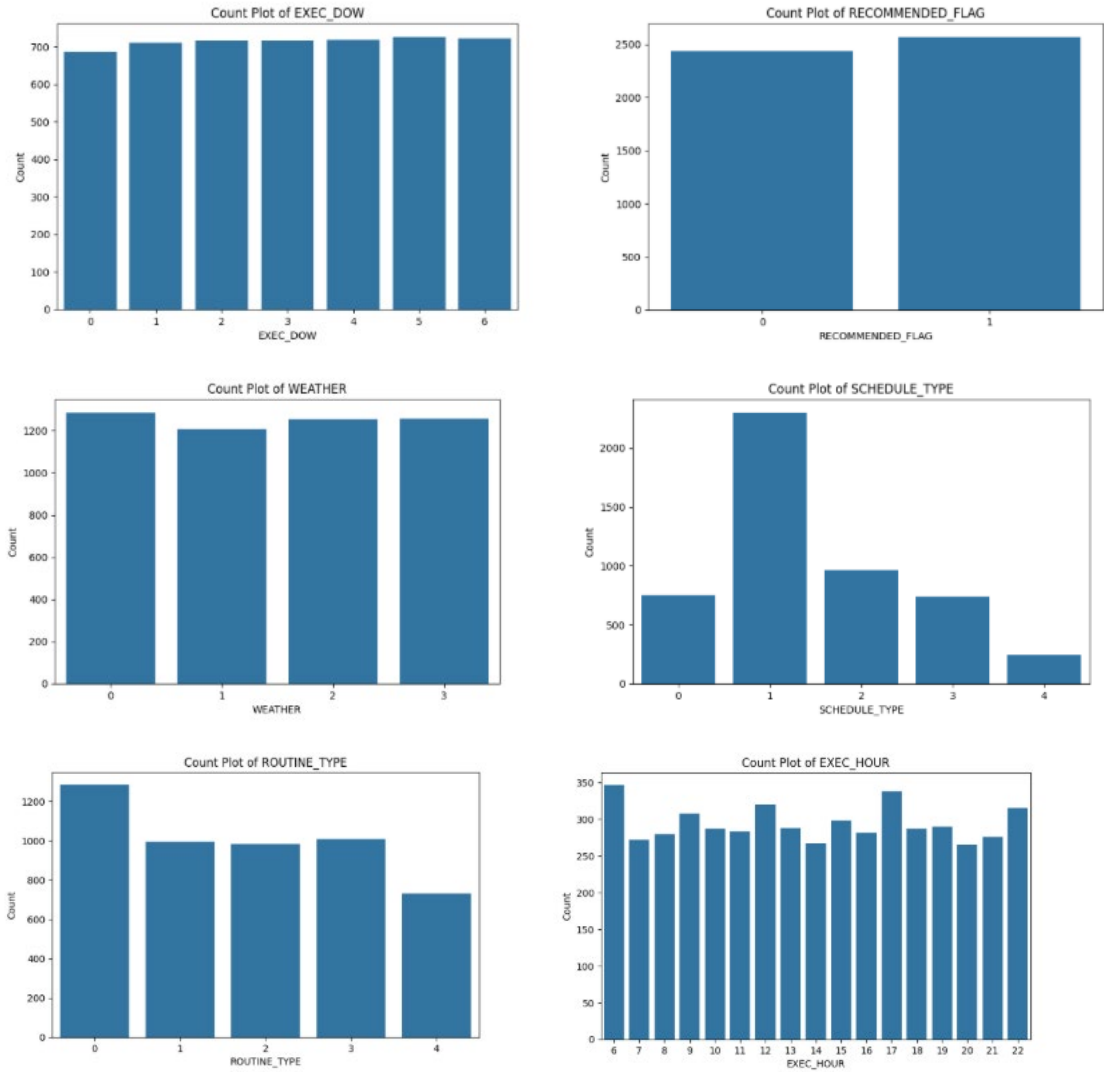
수집 데이터

로그데이터를 참고해 5,000건의 학습용 더미데이터 구축함

- 더미 데이터가 잘 생성되었는지 확인하고 통계적 검증을 위해 분포를 확인

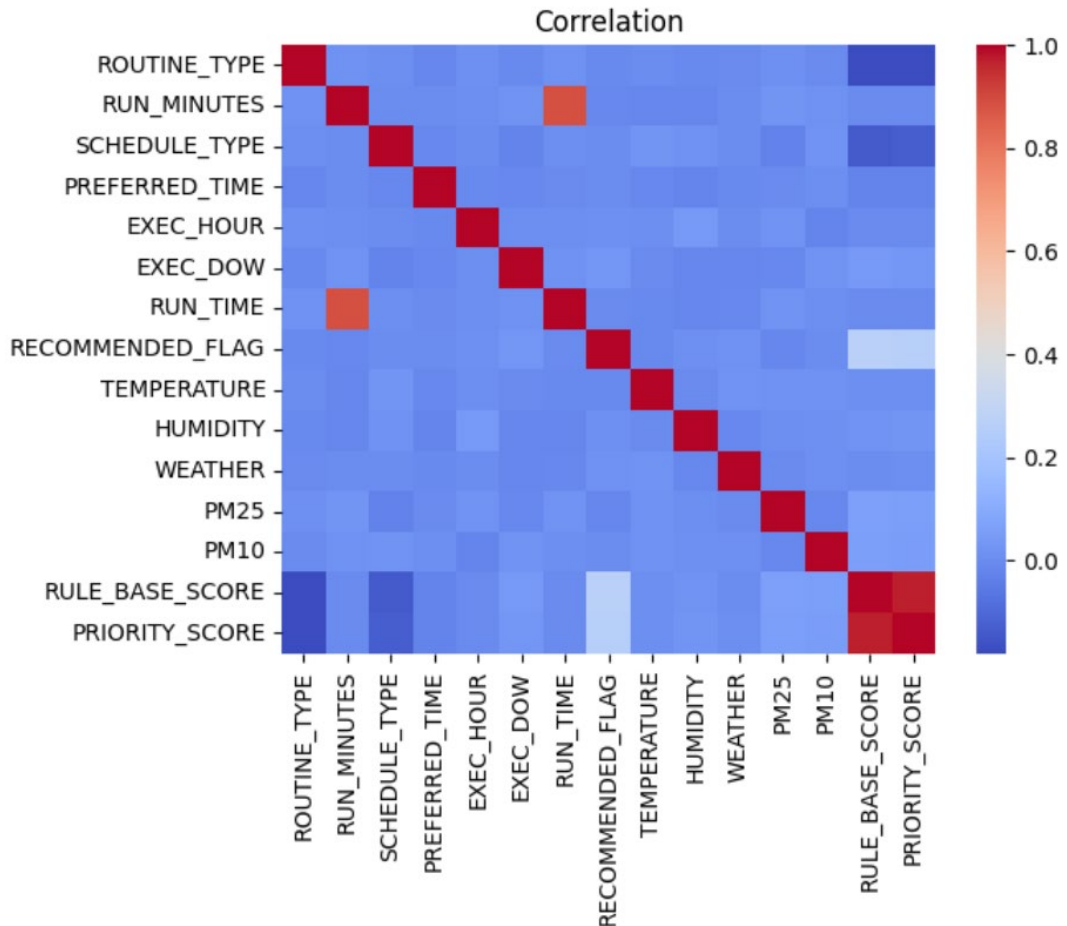
히스토그램 & Bar Plot





생성된 더미데이터의 피처별 분포를 확인한 결과,
대부분의 데이터들이 정규분포와 같은 자연스럽게 퍼진 형태를 보이며,
비정상적인 이상치(outlier)는 관측되지 않았다.
전체적으로 각 특성(feature)들이 의도한 통계적 성질을 유지하며
모델 학습에 적합한 구조를 갖추고 있음을 확인하였다.

상관 관계 확인 결과



외부 환경 요인이 루틴 실행 특성에 과도하게 영향을 주지 않도록 안정적으로 생성음을 알 수 있다.

RUN_MINUTES와 RUN_TIME은 양의 상관관계를 나타내어

실제 사용자 행동(예상시간이 길수록 실제 실행시간도 길어지는 패턴)을 잘 표현 하고 있다.

또한, 주요 범주형 변수는 타겟 변수(PRIORITY_SCORE)에 적절한 수준의 약한 상관을 나타내며

규칙 기반 점수 설계의 의도(과도한 편향 없이 부드러운 우선순위)를 충실히 반영하는 것으로 확인되었다.

피쳐 중요도

	<table border="1"> <caption>Feature Importance Data</caption> <thead> <tr> <th>feature</th> <th>importance</th> </tr> </thead> <tbody> <tr><td>SCHEDULE_TYPE</td><td>0.29</td></tr> <tr><td>RECOMMENDED_FLAG</td><td>0.19</td></tr> <tr><td>PREFERRED_TIME</td><td>0.13</td></tr> <tr><td>EXEC_HOUR</td><td>0.09</td></tr> <tr><td>ROUTINE_TYPE</td><td>0.08</td></tr> <tr><td>PM10</td><td>0.04</td></tr> <tr><td>PM25</td><td>0.04</td></tr> <tr><td>HUMIDITY</td><td>0.03</td></tr> <tr><td>RUN_TIME</td><td>0.02</td></tr> <tr><td>TEMPERATURE</td><td>0.02</td></tr> <tr><td>EXEC_DOW</td><td>0.02</td></tr> <tr><td>RUN_MINUTES</td><td>0.02</td></tr> <tr><td>WEATHER</td><td>0.02</td></tr> </tbody> </table> 루틴 스케줄(SCHEDULE_TYPE), 실행 패턴(RUN_TIME, RECOMMENDED_FLAG) , 선호 시간(PREFERRED_TIME), 사용자의 현재 시간대(EXEC_HOUR), 날씨 패턴(PM10, PM25 등) 순으로 중요도가 나타났다.	feature	importance	SCHEDULE_TYPE	0.29	RECOMMENDED_FLAG	0.19	PREFERRED_TIME	0.13	EXEC_HOUR	0.09	ROUTINE_TYPE	0.08	PM10	0.04	PM25	0.04	HUMIDITY	0.03	RUN_TIME	0.02	TEMPERATURE	0.02	EXEC_DOW	0.02	RUN_MINUTES	0.02	WEATHER	0.02
feature	importance																												
SCHEDULE_TYPE	0.29																												
RECOMMENDED_FLAG	0.19																												
PREFERRED_TIME	0.13																												
EXEC_HOUR	0.09																												
ROUTINE_TYPE	0.08																												
PM10	0.04																												
PM25	0.04																												
HUMIDITY	0.03																												
RUN_TIME	0.02																												
TEMPERATURE	0.02																												
EXEC_DOW	0.02																												
RUN_MINUTES	0.02																												
WEATHER	0.02																												
2. 전처리																													
전처리 과정	해당 데이터는 학습용 더미 데이터이므로 규칙기반의 라벨링을 진행했다. 구간화를 통해 시간대의 특징을 나타냈으며 사용자 패턴을 추출했다.																												
전처리 데이터	더미데이터 5,000건																												
3. 데이터 분석																													
데이터 분석 분석 목표 데이터 분석 시나리오	다음 루틴을 추천하고 최적화하기 위한 우선순위 점수 예측 모델을 제작한다.																												
	- 실제 서비스 환경에서 수집될 데이터를 가정하여 더미데이터 5,000건을 생성하였다. - 스케줄 유형, 선호 시간대, 실행 이력, 추천 여부, 환경 정보(날씨,미세먼지 등)와 같은 다양한 사용자, 환경 피처를 포함하도록 구성 - 생성된 더미데이터에는 규칙 기반 점수(Rule-Based Score)를 적용하였다. - 각 루틴이 현재 시점에서 어느 정도 우선순위를 가져야 하는지를 라벨로 부여 - 사용자의 현재 시간대(EXEC_HOUR), 루틴 스케줄(SCHEDULE_TYPE), 선호 시간(PREFERRED_TIME)과 실행 패턴(RUN_TIME, RECOMMENDED_FLAG) 등을 고려해 우선																												

	순위 점수(PRIORITY_SCORE)를 예측하는 XGBoost 회귀 모델을 제작하고 학습하였다. 4. 학습 결과 예측된 점수를 통해 다음에 실행할 루틴의 추천 순서를 결정하였다. - 분석 결과는 루틴 추천 엔진에 적용되어 사용자의 일상 상황 변화(시간대, 환경 요소, 실행 이력 등)에 따라 실시간으로 적응하는 개인화된 루틴 추천 서비스를 제공하는 데 활용 추후 고도화 방안 초기 버전에서는 규칙 기반 가중치와 Boosting 회귀 모델을 활용하여 루틴 우선순위 점수를 산출하였으나, 데이터가 축적될수록 아래와 같은 방식으로 고도화가 가능하다. 1. 실제 사용자 행동 기반의 고도화된 supervised 모델 - LightGBM, CatBoost 등 다양한 Boosting 기법 적용을 통해, 사용자별 모델(personalized model)로 고도화 가능 2. 강화학습(Reinforcement Learning) 기반 모델 - 사용자 만족도/편의성/실행률 등을 보상 함수로 정의, 루틴 순서, 스케줄링, 가전 조작을 포함한 통합 최적화 가능 3. LLM 기반의 하이브리드 의사결정 모델 - 규칙 기반·ML 기반 예측을 보완하고, 사용자 컨텍스트와 자연어 입력을 이해하는 고도화 모델로 확장
데이터 분석 결과	루틴의 우선 순위 점수를 예측 할 수 있었으며, 서비스를 통해 데이터가 입력되면 모델이 점수를 구해 루틴의 우선 순위를 정할 수 있게 되었다.
4. 모델 생성 및 학습	
모델링 목표	다음 루틴을 추천하고 최적화하기 위한 우선순위 점수를 예측한다. XGBoost 사용 이유 추후 서비스를 통해 쌓일 데이터에는 다양한 범주형·연속형 특성이 혼재하고 규칙 기반 점수를 포함한 구조적 패턴이 존재하기 때문에 트리 기반 앙상블 모델 중 성능과 안정성이 검증된 XGBoost를 사용하였다. XGBoost는 비선형 관계 모델링에 강하고, 피쳐 중요도를 명확히 제공하며, 적은 양의 데이터에서도 높은 예측 성능을 보이며 모델 해석이 용이하다는 장점을 가진다.

학습 모델	또한, Feature 간 상호작용을 자동으로 학습하며, 규칙 기반 점수로 생성된 target(PRIORITY_SCORE)의 패턴을 복잡한 전처리 없이도 자연스럽게 복원할 수 있다. 그렇기 때문에 더미데이터의 규모($N \approx 5,000$)가 크지 않은 상황에서도 과적합을 방지하고 안정적인 성능을 보여 프로토타입 단계에서 사용하기에 가장 적합하고 모델의 목표가 예측값보다 '정확한 우선순위(순서)' 산출에 있기 때문에 트리 기반 모델이 가지는 구조가 최적의 선택이라고 판단했다.
	초기 파라미터 설정 이후, 파라미터를 조정하며 모델을 반복해 학습시켜 최적의 파라미터를 선정했다. 다음은 가장 적합하다고 생각되는 파라미터이다. <pre>n_estimators=500, learning_rate=0.05, max_depth=6 subsample=0.8, colsample_bytree=0.8, objective="reg:squarederror", eval_metric="rmse", random_state=42, n_jobs=-1</pre>
학습 방법	1. Train 데이터로는 모델을 학습하고 Validation 데이터로 검증 2. 파라미터 탐색은 Validation 성능을 기준으로 단계적으로 수행하며 최적값 도출 3. 최종 성능 평가는 Test 데이터셋을 사용해 RMSE, MAE 기준으로 진행
모델 예측 결과	<pre>Train: (3499, 13) Valid: (751, 13) Test: (750, 13) [Train] RMSE: 0.500, MAE: 0.380 [Valid] RMSE: 0.903, MAE: 0.702 [Test] RMSE: 0.878, MAE: 0.699</pre>
분석 검증 방안	5. 검증
	1. 분포 검증·상관분석·규칙 기반 라벨 검증·학습/검증 데이터 분리·Feature Importance 분석을 통해 모델이 규칙 기반 점수 패턴을 안정적으로 학습하고 실제 서비스 흐름에 적용 가능한지 검증했다. 2. 노이즈(0.5 std)를 추가하여 모델이 규칙의 패턴을 학습하면서도 과도한 결정 경계를 형성하지 않도록 안정성을 확보했다.

4. 예측 점수는 절대값보다 순위 기반 의사결정이 핵심이므로
동일 시점에 대해 모델이 일관된 우선순위를 산출하는지 사례 테스트를 수행하였다.

해당 결과는 입력된 데이터를 모델을 통해 예측한 점수대로 우선 순위를 구해 순서대로 출력한 결과이다.

	핵심 피처의 차이에 따라 우선순위가 달라지는 패턴을 명확히 보이기 때문에 분석 목적에 적합하다고 판단된다.
분석 평가 결과	훈련 데이터셋의 RMSE는 0.5, 검증 데이터셋의 RMSE는 0.903, 시험 데이터의 RMSE는 0.878로 측정되었다. 모델은 규칙 기반 점수에 평균 0.5 수준의 노이즈를 추가하여 학습되었기 때문에 검증·테스트 RMSE가 노이즈 크기와 유사한 범위에 있는 것은 과적합 없이 규칙 기반 점수 패턴을 안정적으로 재현하고 있음을 의미한다. 또한 분석의 목적이 '절대값 예측 정확도'가 아니라 상대적 우선순위(랭킹) 산출에 있으므로 현 수준의 RMSE는 모델 설계 의도에 부합하는 적절한 성능으로 판단된다.